

Apunte Física Computacional II

Integrantes: Joaquín Parra
Ignacio Falcón
Alvaro Osses Nieto
Benjamin Fuentes

Resumen

El siguiente documento presenta un apunte para el curso de Física Computacional II, basado en las notas de clase y presentaciones del ramo dictado el profesor Roberto Navarro durante el segundo semestre de los años 2024 y 2025, las notas de clase redactadas por Pedro Contreras y los apuntes de Laboratorio I del profesor Guillermo Rubilar.

Índice de Contenidos

1. Cálculo Numérico	1
1.1. Errores Computacionales	1
1.2. Derivadas numéricas	2
1.2.1. Otros métodos de derivación numérica	4
1.2.2. Derivadas para datos discretos	4
2. Breve introducción a Python	6
2.1. Tipos de variables	7
2.2. Operaciones con listas	7
2.3. Numpy	8
3. Ecuaciones Diferenciales Ordinarias	9
3.1. Método de Euler	9
3.1.1. Definición	9
3.1.1. Ejemplo	9
3.2. Método del salto de la rana	11
3.2.1. Motivación	11
3.2.1. Ejemplo	11
3.2.2. Definición: Esquemas de 2 y 3 Pasos	12
3.2.2. Ejemplo	14
3.3. Métodos Runge-Kutta	16
3.3.1. Métodos de orden superior	17
3.3.2. Formulación semi-general	18
3.3.3. Implementación en Python	19
3.3.4. Teorema II de Buckingham	19
3.4. Comentarios Sobre Estabilidad	20
3.4.1. Motivación	20
3.4.1. Criterio de la Derivada	21
3.4.2. Sistemas Autónomos*	21
3.4.3. Funciones de Lyapunov*	21
4. Introducción a la Búsqueda de Ceros	22
4.1. Método de la Bisección	23
4.2. Método de Newton	24
4.3. Método de la Secante	26
5. Interpolación y ajustes	28
5.1. Interpolación de Lagrange	28
5.2. Matriz de Vandermonde	29
5.2.1. Casos particulares interesantes	30

5.3.	Splines	30
5.4.	Ajustes	32
5.4.1.	Regresión Lineal	32
5.4.2.	Método de mínimos cuadrados ponderado	35
6.	Integración numérica	36
6.1.	Introducción	36
6.2.	Método Newton-Cotes	36
6.3.	Regla del rectángulo	36
6.4.	Regla del trapecioide	37
6.5.	Regla de Simpson	37

Índice de Figuras

1.	Error de la derivada adelantada de $f(x) = \sin(x)$ en $x = 0.3$ para distintos valores de h	3
2.	Soluciones al problema del resorte colgando, obtenidas de forma analítica y con el método de Euler.	11
3.	Gráficos de energía y espacio de fases para el método de Euler.	12
4.	Soluciones obtenidas por el método leap-frog de 3 pasos / velocity Verlet para el resorte colgando.	15
5.	Espacio de fases del oscilador armónico resuelto con RK4.	18
6.	Conjunto de puntos interpolados por Lagrange y función original.	29
7.	Fenómeno de Gibbs causado por interpolación de Lagrange.	31
8.	Ajuste lineal para distribución de puntos.	34

Índice de Tablas

1.	Tipos de variables en <code>python</code>	7
2.	Variables y sus dimensiones	19

Índice de Códigos

1.	Suma de dos decimales en Python	1
2.	Output	1
3.	Easter egg	6
4.	Zen de Python	6
5.	Operaciones de listas de <code>python</code>	7
6.	Ejemplos de comportamiento de arreglos de numpy	8
7.	Algoritmo básico del método de Euler.	10
8.	Algoritmo básico del método leap-frog.	13

9.	Esquema de 3 pasos para el salto de rana.	14
10.	Esquema de tres pasos para el resorte colgando.	14
11.	Resolución del sistema lineal (104) en python.	33

1. Cálculo Numérico

1.1. Errores Computacionales

Si usted ha intentado realizar una suma de dos números decimales mediante algún lenguaje de programación, podrá haber notado que el resultado no es exactamente correcto

Código 1: Suma de dos decimales en Python

```
1 a = 0.1
2 b = 0.2
3 c = 0.3
4
5 print(f"{a + b == c}")
6 print(f"{a + b}")
```

Cuyo output será

Código 2: Output

```
1 False
2 0.30000000000000004
```

Este error se debe a la manera en que las computadoras deben representar números. En general, los humanos representamos los números en base 10, por ejemplo, escribimos el 325 como

$$325 = 3 \cdot 10^2 + 2 \cdot 10^1 + 5 \cdot 10^0 = [325]_{10}$$

Pero los computadores utilizan bits (0 y 1), por lo que estos representan números en base 2, es decir:

$$325 = 2^8 + 2^6 + 2^2 + 2^0 = [101000101]_2$$

Ahora, para representar número reales en el sistema binario, se deben incluir exponentes negativos a la suma de exponentes. En la representación binaria, los números se separan por un exponente y una parte fraccionaria (mantisa) las cuales son separadas por un punto (.). Por ejemplo,

$$\begin{aligned}[2.145]_{10} &= 2 \cdot 10^0 + \frac{1}{8} \\ &= 2^1 + 2^{-3} \\ &= [10.001]_2 \\ &= [1.0001]_2 \cdot 2^1\end{aligned}$$

Este ejemplo en particular funciona bien porque la mantisa del número decimal resulta ser un exponente de 2, veamos que ocurre para un número más complejo:

$$\begin{aligned}
[0.1]_{10} &= [1]_2/[1010]_2 \\
&= [0.00011]_2 \\
&= [0.00011001100110011 \dots] \\
&= 2^{-4} + 2^{-5} + 2^{-8} + 2^{-9} + 2^{-12} + 2^{-13} + \dots \\
&= 2^{-4} \cdot (1 + 2^{-1} + 2^{-4} + 2^{-5} + \dots)
\end{aligned}$$

Obtenemos una serie infinita de potencias de dos. Como en la practica los computadores tienen memoria finita es necesario truncar la serie. En general, la mantisa recibe 52 bits, por lo que la serie se trunca hasta el termino 2^{-52} . en efecto:

$$[0.1]_{10} = 2^{-4} \cdot (1 + 2^{-1} + 2^{-4} + 2^{-5} + \dots + 2^{-52})$$

Este número no será *exactamente* igual a 0.1, si no que será igual a $0.1 \cdot (1 + \epsilon)$. Este será el caso para la gran mayoría de los calculos hechos computacionalmente, el resultado final tendrá una pequeña divergencia con respecto al valor real, es decir

$$\bar{a} = a \cdot (1 + \epsilon)$$

donde $\bar{a}$ y a son los valores calculado y real, respectivamente. La diferencia entre el número deseado y el resultante se denomina **error computacional**. Dado que las computadores no pueden físicamente lidiar con cantidades infinitesimalmente pequeñas, muchos de los métodos de cálculo adaptados a la programación que veremos a lo largo del apunte serán basados en aproximaciones, por esta razón es importante aprender a identificar fuentes de errores numericos y sus ordenes de magnitud.

1.2. Derivadas numéricas

Sabemos del calculo diferencial que la derivada de una función arbitraria f evaluada en un punto arbitrario x_0 es dada por

$$f'(x_0) = \lim_{h \rightarrow 0} \frac{f(x_0 + h) - f(x_0)}{h} \quad (1)$$

pero ya sabemos que necesitamos que h sea un número finito, por lo que estamos obligados a seguir anaizando esta definición. De la serie de taylor de f evaluada en un punto $a + h$, $h > 0$, tenemos que:

$$f(a + h) = f(a) + f'(a)h + \frac{1}{2!}f''(a)h^2 + \frac{1}{3!}f'''(a)h^3 + \dots \quad (2)$$

Podemos reemplazar los terminos de orden superior por un $f(\xi)h^2$, $a \leq \xi \leq a + h$ (Teorema del valor medio), luego

$$f(a + h) = f(a) + f'(a)h + \frac{1}{2!}f''(\xi)h^2 \quad (3)$$

$$f'(a) = \frac{f(a + h) - f(a)}{h} - \frac{1}{2!}f''(\xi)h \quad (4)$$

Aquí, hemos obtenido una expresión finita para la derivada de una función evaluada en un punto, denominada *derivada adelantada*. Ahora, si buscamos el error de la ecuación (4) vemos que

$$Error = \left| f'(a) - \frac{f(a + h) - f(a)}{h} \right| = \frac{1}{2!} |f''(\xi)| h \quad (5)$$

en notación $\mathcal{O}$ de Landau,

$$Error = \frac{1}{2!} |f''(\xi)| h = \mathcal{O}(h) \quad (6)$$

Esto quiere decir que el error de truncamiento de la derivada adelantada será del orden de h , como se puede observar en la figura 1, el error de la derivada decrece linealmente con el orden de magnitud de h hasta aproximadamente $h \sim 10^{-8}$, pasado ese punto h se vuelve demasiado pequeño para la computadora y predomina el error computacional. Por esta razón, a la hora de implementar este, o cualquiera de los métodos expuestos en este apunte, es conveniente utilizar un $h \sim 10^{-5}$.

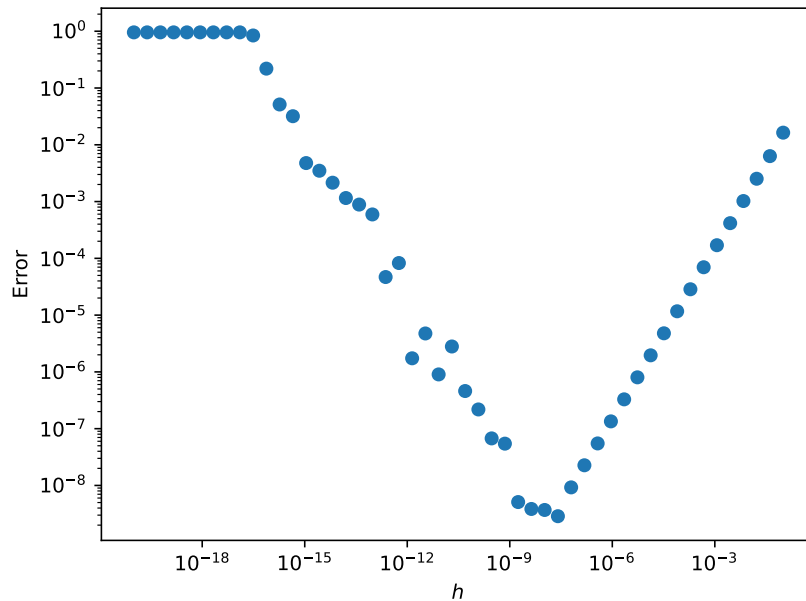


Figura 1: Error de la derivada adelantada de $f(x) = \sin(x)$ en $x = 0.3$ para distintos valores de h

1.2.1. Otros métodos de derivación numérica

Si bien la derivada adelantada es útil debido a lo simple de su implementación, muchas veces queremos utilizar métodos cuyo error sea de un mayor orden de h , es decir, menor.

Similarmente a como obtuvimos la expresión (4), podemos obtener una derivada **retrasada**, en efecto, consideramos la serie de Taylor de $f(a - h)$,

$$f(a - h) = f(a) - f'(a)h + \frac{1}{2!}f''(\xi)h^2 \quad (7)$$

$$f'(a) = \frac{f(a) - f(a - h)}{h} + \frac{1}{2!}f''(\xi)h \quad (8)$$

Si consideramos las series de Taylor de $f(a + h)$ y $f(a - h)$ expandidas hasta el tercer orden,

$$f(a + h) = f(a) + f'(a)h + \frac{1}{2!}f''(a)h^2 + \frac{1}{3!}f'''(\xi)h^3 \quad (9)$$

$$f(a - h) = f(a) - f'(a)h + \frac{1}{2!}f''(a)h^2 - \frac{1}{3!}f'''(\xi)h^3 \quad (10)$$

Ahora si restamos obtenemos,

$$f(a + h) - f(a - h) = 2f'(a)h + \frac{2}{3!}f'''(\xi)h^3 \quad (11)$$

$$2f'(a)h = f(a + h) - f(a - h) - \frac{2}{3!}f'''(\xi)h^3 \quad (12)$$

$$f'(a) = \frac{f(a + h) - f(a - h)}{2h} - \frac{1}{6}f'''(\xi)h^3 \quad (13)$$

Así, obtenemos una expresión para la **derivada centrada**, con error del orden de h^3 .

Podemos obtener muchos otros métodos de derivación numérica por medios similares, en general el proceso consiste en jugar con series de Taylor hasta poder despejar el término $f'(a)$.

1.2.2. Derivadas para datos discretos

En general, uno no suele tratar con funciones definidas analíticamente, si no que tendremos conjuntos de datos relacionados a alguna magnitud física, si quisieramos eventualmente encontrar la derivada de la "función" que definen estos datos, debemos aprender a tratar con puntos no necesariamente equidistantes.

Similarmente a caso anterior, dado un conjunto de puntos x_k y una función continua y diferenciable tal que $f(x_k) = f_k$, definimos la distancia entre dos puntos como $h_k = x_k - x_{k-1}$. Podemos obtener una primera aproximación simplemente tomando la pendiente entre dos puntos, esto es

$$\frac{df_k}{dx_k} \approx \frac{f_k - f_{k-1}}{x_k - x_{k-1}} \quad (14)$$

Esto es analogo a la derivada atrasada encontrada en la sección anterior, y se puede demostrar

que tendrá un error del orden de h_k . Notar además que si imponemos que los nodos estén igualmente espaciados, es decir, $x_k - x_{k-1} = h = cte.$, la ecuación (14) recupera la ecuación (8).

Ahora, estamos incentivados a encontrar un método de derivación discreta con error de menor orden, para ello consideramos la serie de taylor de f_{k+1} y f_{k-1}

$$f_{k+1} = f_k + \frac{df_k}{dx}h_{k+1} + \frac{1}{2!}\frac{d^2f_k}{dx^2}h_{k+1}^2 + \frac{1}{3!}\frac{d^3f(\xi_1)}{dx^3}h_{k+1}^3 \quad (15)$$

$$f_{k-1} = f_k - \frac{df_k}{dx}h_k + \frac{1}{2!}\frac{d^2f_k}{dx^2}h_k^2 - \frac{1}{3!}\frac{d^3f(\xi_2)}{dx^3}h_k^3 \quad (16)$$

con $x_{k-1} \leq \xi_2 \leq x_k \leq \xi_1 \leq x_{k+1}$. Despejando $\frac{df_k}{dx}$ en ambas expresiones obtenemos,

$$\frac{df_k}{dx}h_{k+1} = f_{k+1} - f_k - \frac{1}{2!}\frac{d^2f_k}{dx^2}h_{k+1}^2 - \frac{d^3f(\xi_1)}{dx^3}h_{k+1}^3 \quad (17)$$

$$\frac{df_k}{dx}h_k = f_k - f_{k-1} + \frac{1}{2!}\frac{d^2f_k}{dx^2}h_k^2 + \frac{d^3f(\xi_2)}{dx^3}h_k^3 \quad (18)$$

Multiplicando (17) y (18) por h_k/h_{k+1} y h_{k+1}/h_k , respectivamente,

$$\frac{df_k}{dx}h_k = \frac{(f_{k+1} - f_k)h_k}{h_{k+1}} - \frac{1}{2!}\frac{d^2f_k}{dx^2}h_kh_{k+1} - \frac{d^3f(\xi_1)}{dx^3}h_{k+1}^2h_k \quad (19)$$

$$\frac{df_k}{dx}h_{k+1} = \frac{(f_k - f_{k-1})h_{k+1}}{h_k} + \frac{1}{2!}\frac{d^2f_k}{dx^2}h_kh_{k+1} + \frac{d^3f(\xi_2)}{dx^3}h_k^2h_{k+1} \quad (20)$$

Sumando,

$$[h_{k+1} + h_k]\frac{df_k}{dx} = \frac{(f_{k+1} - f_k)h_k}{h_{k+1}} + \frac{(f_k - f_{k-1})h_{k+1}}{h_k} + \frac{1}{3!}[f'''(\xi_2)h_k - f'''(\xi_1)h_{k+1}]h_kh_{k+1} \quad (21)$$

Finalmente,

$$\frac{df_k}{dx} = \frac{(f_{k+1} - f_k)}{h_{k+1} + h_k} \frac{h_k}{h_{k+1}} + \frac{(f_k - f_{k-1})}{h_{k+1} + h_k} \frac{h_{k+1}}{h_k} + \frac{1}{3!} \left[\frac{f'''(\xi_2)h_k - f'''(\xi_1)h_{k+1}}{h_{k+1} + h_k} \right] h_kh_{k+1} \quad (22)$$

Ahora, por el teorema del valor medio, podemos reemplazar el último termino entre corchetes por una función evaluada en un punto ξ , con $\xi_2 \leq \xi \leq \xi_1$, en efecto,

$$\frac{df_k}{dx} = \frac{(f_{k+1} - f_k)}{h_{k+1} + h_k} \frac{h_k}{h_{k+1}} + \frac{(f_k - f_{k-1})}{h_{k+1} + h_k} \frac{h_{k+1}}{h_k} + \frac{1}{3!} \frac{d^3f_k(\xi)}{dx^3} h_kh_{k+1} \quad (23)$$

$$= \frac{(f_{k+1} - f_k)}{h_{k+1} + h_k} \frac{h_k}{h_{k+1}} + \frac{(f_k - f_{k-1})}{h_{k+1} + h_k} \frac{h_{k+1}}{h_k} + \mathcal{O}(h_kh_{k+1}) \quad (24)$$

Así, logramos encontrar una expresión para la derivada de una función definida punto

a punto con nodos no necesariamente equidistantes, con error de orden cuadrático. Puede comprobar que si imponemos que los nodos sean equidistantes ($h_k = h_{k+1} = h$), se recupera la expresión (13) para la derivada centrada (**Demuestrelo!**). Más adelante estudiaremos métodos más avanzados para tratar con distribuciones de datos con interpolaciones y splines. Para terminar, cabe mencionar que la mayor desventaja de este método es que, al utilizar 3 puntos, aplicándola a una serie de puntos, la expresión numérica de la derivada tendrá dos puntos menos que la serie original. Por esta razón, en la práctica se implementa una derivada adelantada para los puntos iniciales de un set de datos, y una atrasada para los finales; la derivada centrada se utiliza solo para los puntos intermedios.

2. Breve introducción a Python

Python es un lenguaje de alto nivel con una sintaxis enfocada en la legibilidad y la simpleza. Los principios fundamentales del lenguaje están codificados *dentro* del mismo, pruebe ejecutar el código 3

Código 3: Easter egg

```
1 import this
```

Spoiler: Si lo hace, el output será el **Zen de Python**

Código 4: Zen de Python

```
1 Beautiful is better than ugly.
2 Explicit is better than implicit.
3 Simple is better than complex.
4 Complex is better than complicated.
5 Flat is better than nested.
6 Sparse is better than dense.
7 Readability counts.
8 Special cases aren't special enough to break the rules.
9 Although practicality beats purity.
10 Errors should never pass silently.
11 Unless explicitly silenced.
12 In the face of ambiguity, refuse the temptation to guess.
13 There should be one-- and preferably only one --obvious way to do it.
14 Although that way may not be obvious at first unless you're Dutch.
15 Now is better than never.
16 Although never is often better than *right* now.
17 If the implementation is hard to explain, it's a bad idea.
18 If the implementation is easy to explain, it may be a good idea.
19 Namespaces are one honking great idea -- let's do more of those!
```

2.1. Tipos de variables

Tabla 1: Tipos de variables en python

Variable	Tipo
<code>x = "holi"</code>	<code>str</code> Cadena de caracteres
<code>x = 14</code>	<code>int</code> Número entero
<code>x = 6.75</code>	<code>float</code> Número real (punto flotante)
<code>x = 2 + 1j</code>	<code>complex</code> Número complejo
<code>x = ["one", 2, 3]</code>	<code>list</code> Lista de elementos
<code>x = ("uno", 2, 3.0)</code>	<code>tuple</code> Tupla de elementos
<code>x = {"nombre": "Juan", "edad": 22}</code>	<code>dict</code> Diccionario
<code>x = 1, 2, 3</code>	<code>set</code> Conjunto
<code>x = True</code>	<code>bool</code> Tipo lógico (booleano)
<code>x = None</code>	<code>NoneType</code> Tipo Nulo

2.2. Operaciones con listas

En la práctica, a menudo nos encontraremos obligados a trabajar con listas de python, por esta razón, es importante conocer las distintas maneras en que podemos extraer, añadir o eliminar elementos de estas.

Código 5: Operaciones de listas de python

```

1  lista_ex = [7, 2, 5, 9] #Lista de ejemplo
2
3  listacat = lista_ex + [1, 6] #concatena elementos : [7, 2, 5, 9, 1, 6]
4  listacat.append(5) #añade elemento al final de la lista : [7, 2, 5, 9, 1, 6, 5]
5  listacat.pop() #elimina el último elemento : [7, 2, 5, 9, 1, 6]
6
7  listacat[0] #accede al primer elemento (elemento 0) : 7
8  listacat[-1] #accede al último elemento : 6
9  listacat[27] #tira error IndexError : la lista solo tiene 6 elementos
10
11 listacat[1:3] #accede elementos desde listacat[1] hasta listacat[2] : [2, 5]
12 listacat[1:] #accede elementos desde listacat[1] hasta el final : [2, 5, 9, 1, 6]
13 listacat[:-1] #accede elementos desde listacat[0] hasta el penultimo elemento
14 listacat[::2] #recorre toda la lista con saltos de dos elementos : [7, 5, 1]
15 listacat[::-1] #recorre la lista en orden inverso : [6, 1, 9, 5, 2, 7]
16

```

En general, la notación para recorrer listas es de la forma `lista[desde:hasta:paso]`, notando que el elemento `hasta` es excluyente, es decir, `lista[0:5]` recorrerá desde el elemento 0 hasta el

elemento 4, si tocar el elemento 5.

2.3. Numpy

numpy Es una librería de python que introduce varios elementos y operaciones matemáticas útiles para implementar los métodos presentados más adelante. Una adición importante son los **arreglos** de numpy, a grandes rasgos, estos pueden comportarse como vectores o matrices, con la diferencia que todas las operaciones las realiza **elemento a elemento**.

Código 6: Ejemplos de comportamiento de arreglos de numpy

```
1  #Para importar numpy
2  import numpy as np
3
4  v = np.array([1, 2, 3, 4]) #vector de 4 elementos
5
6  type(v) #retorna <class "numpy.ndarray">
7  v.dtype #retorna el tipo de los elementos de v : int64
8
9  #los arreglos de numpy solo pueden almacenar un tipo de elementos a la vez
10 #esto los diferencia de las listas de python que pueden almacenar elementos de
11 #distinto tipo, por ejemplo
12
13 v[0] = 7 + 2j #solo almacenará la parte real del complex: array([7, 2, 3, 4])
14
15 #podemos forzar el tipo de los elemento del array para que pueda guardar complejos
16
17 v = np.array([1, 2, 3, 4], dtype = complex)
18 v[0] = 7 + 2j # array([7 + 2j, 2 + 0j, 3 + 0j, 4 + 0j])
19
```

3. Ecuaciones Diferenciales Ordinarias

3.1. Método de Euler

3.1.1. Definición

El método de Euler es el acercamiento más simple a la resolución numérica de ecuaciones diferenciales ordinarias. Consiste en realizar una simple aproximación en Taylor a primer orden. Dada la simplicidad del método, se abordarán de inmediato los casos en una y varias dimensiones.

Considere que busca resolver una ecuación diferencial ordinaria de orden n . Como recordará de su curso de ecuaciones diferenciales ordinarias, podemos representar este problema como una EDO de primer orden en forma vectorial:

$$\begin{pmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \vdots \\ \dot{x}_n \end{pmatrix} = \frac{d\vec{x}}{dt} = \vec{f}(\vec{x}, t), \quad (25)$$

con $\vec{x} = (x_1(t), x_2(t), \dots, x_n(t))$ un vector de más funciones a encontrar y $\vec{f}$ una función vectorial conocida. En su curso de EDO quizás vio esta formulación con la forma:

$$\begin{pmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \vdots \\ \dot{x}_{n-1} \\ \dot{x}_n \end{pmatrix} = \begin{pmatrix} x_2 \\ x_3 \\ \vdots \\ x_n \\ g(x) \end{pmatrix}, \quad (26)$$

con $g(x)$ generalmente la propia EDO en cuestión. Esta formulación es sumamente útil y será utilizada más adelante. Aproximando (25) como una derivada adelantada, tenemos el **método de Euler**:

$$\vec{x}(t+h) = \vec{x}(t) + h\vec{f}(\vec{x}, t) + \mathcal{O}(h^2). \quad (27)$$

Esto define un método de primer orden. En general, el orden n de un integrador o método se suele expresar como el n tal que los pasos tienen error de orden $\mathcal{O}(h^{n+1})$. Geométricamente, lo que sucede paso a paso es que partimos situados en un punto $\vec{x}_0$ dado por una condición inicial en un tiempo t_0 , y avanzamos al siguiente punto siguiendo el vector tangente $f(\vec{x}_0, t)$. Abordaremos el algoritmo con un problema.

Ejemplo

Consideremos un objeto de masa m colgado de un resorte de constante elástica k , en un campo gravitatorio de magnitud g . Si el origen del sistema de referencia se encuentra ubicado

en el resorte en su longitud natural, entonces la altura $y(t)$ se rige por la ecuación diferencial:

$$y''(t) + \omega^2 y(t) = -g, \quad (28)$$

donde $\omega = \sqrt{k/m}$. Si tenemos las condiciones iniciales $y(0), y'(0)$, la solución analítica es:

$$y(t) = \left[y(0) + \frac{g}{\omega^2} \right] \cos(\omega t) + \frac{y'(0)}{\omega} \sin(\omega t) - \frac{g}{\omega^2}. \quad (29)$$

Como la ecuación diferencial es de segundo orden, podemos expresarla en forma vectorial para reducir el problema a una ecuación diferencial de primer orden. Tenemos:

$$\frac{dy}{dt} = v(t), \quad \frac{dv}{dt} = -\omega^2 y(t) - g, \quad (30)$$

es decir, podemos reescribir el problema como:

$$\vec{x} = \begin{pmatrix} y(t) \\ v(t) \end{pmatrix}, \quad \vec{f}(\vec{x}, t) = \begin{pmatrix} v(t) \\ -\omega^2 y(t) - g \end{pmatrix}, \quad \frac{d\vec{x}}{dt} = \vec{f}(\vec{x}, t). \quad (31)$$

De esta forma, el algoritmo relevante del método es¹:

Código 7: Algoritmo básico del método de Euler.

```

1  def f(x, omega=1.0, g = 9.8): # creamos el vector f, acá hay que definir omega
2      y, v = x # desempaquetamos las variables
3
4      return array([v, -y * omega ** 2 - g])
5
6  x = zeros([Ntiempo, 2]) # Definimos un vector x con los pares (x,v) a cada paso
7  x[0] = [y0, v0] # Imponemos que la primera entrada de x sean las cond. iniciales
8
9
10 for n in range (Ntiempo):
11     x[n+1] = x[n] + h * f(x[n], omega, g) #partiendo de x[0], cambia los pares (0,0) por los
        ↪ valores del método
12 #finalmente, solo hace falta desempaquetar el resultado del vector x.
```

Finalmente, para ver la eficacia del método, se ha simulado tanto la solución analítica como la solución mediante el método de Euler en la figura 2. Podemos apreciar como **rápidamente** la solución numérica difiere de la solución analítica. En particular, vemos como la amplitud de la oscilación va aumentando conforme avanza el tiempo, ¿tiene esto significado físico? La respuesta se aborda en la siguiente sección.

¹ Este código es un esquema. Para ver código real, puede visitar el repositorio.

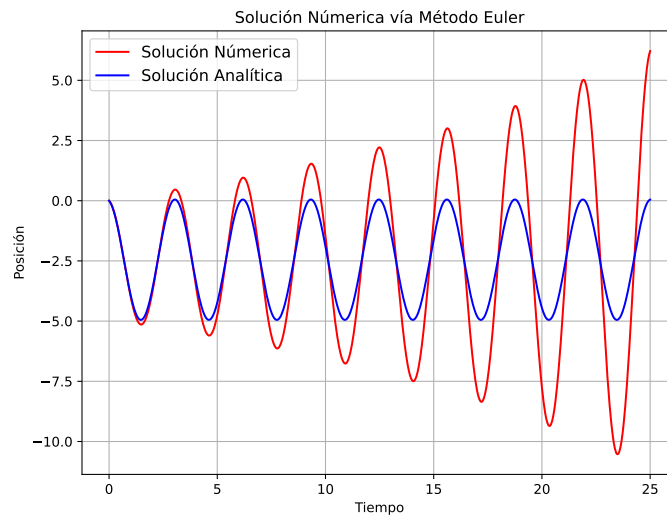


Figura 2: Soluciones al problema del resorte colgando, obtenidas de forma analítica y con el método de Euler.

3.2. Método del salto de la rana

3.2.1. Motivación

Si bien el método de Euler es un approach simple y familiar (pero no muy estable) a la resolución numérica de EDO's, **físicamente** plantea un problema importante: **no conserva la energía del sistema**. Un rápido cálculo nos muestra esto:

Ejemplo

Para un sistema masa-resorte colgando, se puede mostrar que **energía total** del sistema es de la forma:

$$E(t) = \frac{1}{2}mv^2 + \frac{1}{2}m^2\omega^2 \left(y + \frac{g}{\omega^2} \right)^2. \quad (32)$$

Luego, bajo el método de Euler, la energía en tiempos discretos es:

$$\begin{aligned} E_{n+1} &= \frac{1}{2}mv_{n+1}^2 + \frac{1}{2}m^2\omega^2 \left(y_{n+1} + \frac{g}{\omega^2} \right)^2, \\ &= \frac{1}{2}m \left[v_n - h(\omega^2 y_n + g) \right]^2 + \frac{1}{2}m^2\omega^2 \left[y_n + hv_n + \frac{g}{\omega^2} \right]^2, \\ &= (1 + \omega^2 h^2) \left[\frac{1}{2}mv_n^2 + \frac{1}{2}m^2\omega^2 \left(y_n + \frac{g}{\omega^2} \right)^2 \right], \\ &= (1 + \omega^2 h^2) E_n. \end{aligned}$$

Es decir, el método de Euler no conserva la energía pues $E_{n+1} > E_n$ para todo n . Puede ver esto también modelando la ecuación (32), o realizando un gráfico de rapidez versus posición (una especie de espacio de fases) mientras evoluciona el tiempo (ver figura 3). El hecho que la trayectoria no sea cerrada en el espacio de fases es consecuencia directa de la pérdida de energía.

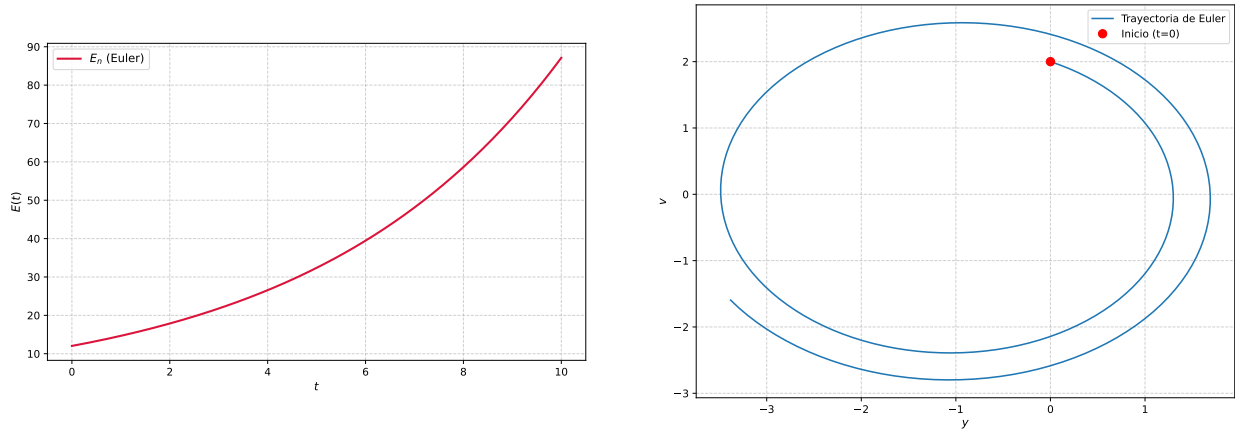


Figura 3: Gráficos de energía y espacio de fases para el método de Euler.

3.2.2. Definición: Esquemas de 2 y 3 Pasos

Este método resulta sumamente útil al momento de lidiar con sistemas en los cuales hay **conservación de la energía mecánica** puesto que su formulación basada en pasos semi-enteros incorpora una estructura simpléctica² (conserva la energía). Este tipo de métodos suelen ser llamados **integradores simplécticos**. El método en cuestión consiste en usar un esquema de derivadas centradas que permiten que, para un paso Δt , se actualice la información de la rapidez dos veces por paso. Una primera formulación del método consiste en definir:

$$v(t + h/2) = \frac{x(t + h) - x(t)}{h} + \mathcal{O}(h^2), \quad (33)$$

$$a(t) = \frac{v(t + h/2) - v(t - h/2)}{h} + \mathcal{O}(h^2), \quad (34)$$

lo que define un integrador de segundo orden con 2 pasos. Multiplicando ambos lados por h , tenemos :

$$v(t + h/2) = v(t - h/2) + ha(t) + \mathcal{O}(h^3), \quad (35)$$

$$x(t + h) = x(t) + hv(t + h/2) + \mathcal{O}(h^3). \quad (36)$$

² La demostración de esto es muy compleja. Para más acerca de integradores simplécticos, puede leer este [artículo](#), que puede encontrar [aquí](#).

Note que el método requiere una condición inicial $v(t_0) = v_0$ a partir del cual computamos los $v_{n+1/2}$ (esto siempre ocurrirá en esquemas basados en derivadas centradas) además de tener información acerca de la aceleración de antemano. Por esto, el esquema suele iniciarse con el método de Euler³:

$$v(h/2) = v(0) + \frac{h}{2}a(0).$$

Bajo estos supuestos, el algoritmo relevante que utiliza el método es

Código 8: Algoritmo básico del método leap-frog.

```

1  # Condiciones iniciales
2  x[0], v[0] = x0, v0
3  # Metodo de Euler para avanzar a t = h/2
4  # Escribimos v_1/2 = v_0 + (h/2)a_0
5  v[0] = v[0] + (h/2) * a[0] # v[0] representa a v_1/2
6
7  # Metodo del salto de rana
8  for n in range(N):
9      x[n+1] = x[n] + h * v[n] #v[n] representa v_n+1/2
10     v[n+1] = v[n] + h * a[n] #v[n+1] representa v_n+3/2
11     ventero[n+1] = 0.5 * (v[n+1] + v[n]) # calcula v_n
12

```

Notemos que bajo este esquema, no tenemos información acerca de la velocidad en tiempos enteros, por lo que generalmente se suele considerar la media aritmética entre 2 velocidades adyacentes (esto se ha implementado en la última línea).

También es útil implementar este método como un esquema de 3 pasos, calculando velocidades tanto a tiempos enteros como semi-enteros. La utilidad de esta formulación es que dejamos de depender del método de Euler para inicializar $v_{1/2}$. A continuación, se muestra el razonamiento para componer el esquema. Consideremos la derivada centrada en $t + h/2$:

$$v(t + h/2) = \frac{x(t + h) - x(t)}{2 \cdot h/2} + \mathcal{O}(h^2), \quad (37)$$

$$\Rightarrow x(t + h) = x(t) + hv(t + h/2) + \mathcal{O}(h^3). \quad (38)$$

Ahora, para la velocidad, consideremos la derivada centrada en t de v :

$$a(t) = v'(t) = \frac{v(t + h/2) - v(t - h/2)}{h} + \mathcal{O}(h^2) \quad (39)$$

$$\Rightarrow v(t + h/2) = v(t - h/2) + ha(t) + \mathcal{O}(h^3). \quad (40)$$

³ Esto no significa que todo el método sea tipo Euler.

Expandiendo $v(t - h/2)$ alrededor de t , tenemos:

$$v(t + h/2) = v(t) - \frac{h}{2}v'(t) + ha(t) + \mathcal{O}(h^3), \quad (41)$$

$$\Rightarrow v(t + h/2) = v(t) + \frac{h}{2}a(t) + \mathcal{O}(h^3). \quad (42)$$

Finalmente, para obtener el término $v(t + h)$ (el paso entero), se considera una expansión a primer orden centrada en $t + h/2$:

$$v(t + h) = v(t + h/2) + \frac{h}{2}a(t + h/2). \quad (43)$$

Las ecuaciones (38), (42) y (43) definen el esquema de 3 pasos del método. En forma de código, el algoritmo relevante es:

Código 9: Esquema de 3 pasos para el salto de rana.

```

1  # condiciones iniciales
2  x[0], v[0] = x0, v0
3
4  # Metodo de salto de rana
5  for n in range(N-1):
6      vmedio = v[n] + (h/2) * a[n] # v[n] representa v_n
7      x[n+1] = x[n] + h * vmedio # vmedio representa v_{n+1/2}
8      v[n+1] = vmedio + (h/2) * a[n+1] # v[n+1] representa v_{n+1}
9

```

Generalmente, al momento de implementar el método, el problema se suele abordar de manera vectorial, y uno resuelve el sistema:

$$\begin{pmatrix} \dot{x} \\ \dot{v} \end{pmatrix} = \begin{pmatrix} v \\ a \end{pmatrix} \quad (44)$$

Ejemplo

Volviendo al problema del resorte colgando que abordamos con el método de Euler, tenemos utilizando el método leap-frog con un esquema de tres pasos, el código base (con algunos agregados) es:

Código 10: Esquema de tres pasos para el resorte colgando.

```

1  def _condiciones_iniciales(t, x0, y0):
2      x0 = np.asarray(x0)
3      y0 = np.asarray(y0)
4      x = np.zeros((len(t),) + x0.shape)
5      y = np.zeros((len(t),) + y0.shape)
6      x[0] = x0

```

```

7   y[0] = y0
8   return x,y
9
10  def dy(a, omega = 2.0, g= 9.8):
11      return -1 * ( omega **2 ) * a - g
12
13  def leapfrog(dy, x0, y0, t, **kwargs):
14      dt = np.diff(t) #paso
15      x, y = __condiciones_iniciales(t, x0, y0)
16      dy0 = dy(x0)
17      for n in range(t.size-1):
18          ymedio = y[n] + 0.5 * dt[n] * dy0
19          x[n+1] = x[n] + dt[n] * ymedio
20          dy0 = dy(x[n+1])
21          y[n+1] = ymedio + 0.5 * dt[n] * dy0
22      return x, y
23
24  t = np.linspace(0,10,1000)
25  u,w = leapfrog(dy, 0.0, -1.0,t, omega=2.0)
26
27  omega = 2.0
28  E = np.zeros(len(t))
29  E[0] = 0.5 * w[0]**2 + 0.5 * omega **2 * (u[0] + 9.8 / (omega ** 2))**2
30
31  for n in range(len(E)-1):
32      # Calcular energía en el paso n+1
33      E[n + 1] = 0.5 * w[n+1]**2 + 0.5 * (omega) **2 * (u[n+1] + 9.8 / (omega**2)) ** 2
34

```

Tenemos las soluciones para el espacio de fases y para la energía en la figura 4. Notemos como la trayectoria (aproximadamente) es cerrada, y que la energía es conservada (aproximadamente, notar que las discrepancias oscilan en las milésimas).

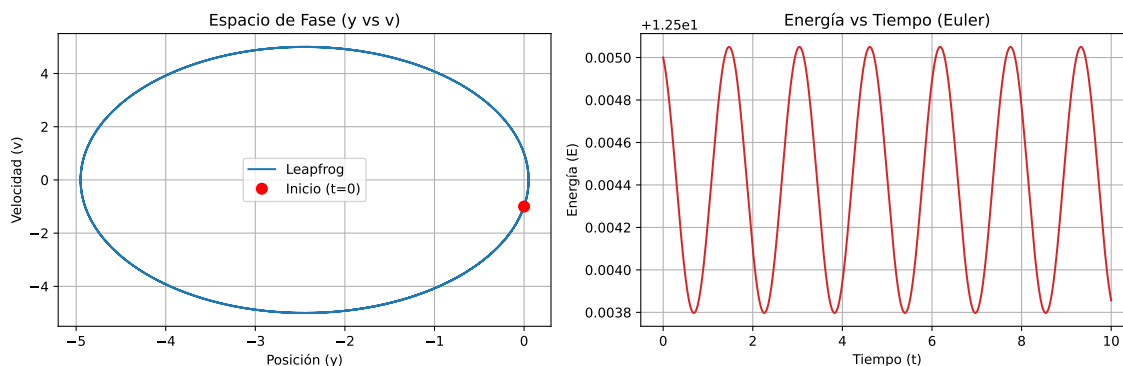


Figura 4: Soluciones obtenidas por el método leap-frog de 3 pasos / velocity Verlet para el resorte colgando.

3.3. Métodos Runge-Kutta

Los métodos de Runge-Kutta son una familia de métodos numéricos iterativos para resolver ecuaciones diferenciales ordinarias, utilizando el valor de la solución en t_0 para calcular la solución en $t_1 = t_0 + h$, y repitiendo este proceso hasta el tiempo final $t_n = t_{n-1} + h$, obteniendo así los valores aproximados de la solución en un intervalo de tiempo discreto y finito. Estos son probablemente los más usados en la física dado su versatilidad y su relativa simpleza, podrá notar que este método es bastante parecido al método de Euler discutido al inicio de este capítulo. Similarmente a los esquemas anteriores, iniciamos con un problema del tipo

$$\frac{dx}{dt} = f(t, x) \quad (45)$$

Podemos aproximar una solución considerando expansiones de $x(t)$ y $x(t+h)$, con $h > 0$, centradas en $t + \frac{h}{2}$, en efecto:

$$\begin{aligned} x(t+h) &= x\left(t + \frac{h}{2}\right) + x'\left(t + \frac{h}{2}\right) \left(\frac{h}{2}\right) + \frac{1}{2}x''\left(t + \frac{h}{2}\right) \left(\frac{h}{2}\right)^2 + \mathcal{O}(h^3) \\ x(t) &= x\left(t + \frac{h}{2}\right) - x'\left(t + \frac{h}{2}\right) \left(\frac{h}{2}\right) + \frac{1}{2}x''\left(t + \frac{h}{2}\right) \left(\frac{h}{2}\right)^2 - \mathcal{O}(h^3) \end{aligned}$$

Restando ambas expresiones obtendremos

$$x(t+h) - x(t) = x'\left(t + \frac{h}{2}\right)h + \mathcal{O}(h^3) \quad (46)$$

Pero, del enunciado del problema, $x'(t + \frac{h}{2}) = f(t + \frac{h}{2}, x(t + \frac{h}{2}))$. De momento parece que no hemos hecho nada, pues solo hemos cambiado el problema de no conocer $x(t)$ a no conocer $x(t + \frac{h}{2})$, pero si recordamos el método de Euler,

$$x\left(t + \frac{h}{2}\right) = x(t) + f(t, x(t))\frac{h}{2} \quad (47)$$

Las expresiones (47) y (46) consituyen el método de Runge-Kutta de orden 2, o simplemente, RK2. En la literatura se suele representar de la forma

$$K_1 = f(t, x(t)) \quad (48)$$

$$K_2 = f\left(t + \frac{h}{2}, x(t) + \frac{h}{2}K_1\right) \quad (49)$$

$$x(t+h) = x(t) + hK_2 \quad (50)$$

Esta **no** es la única forma de construir un RK2, de hecho, existen infinitas formas de hacerlo.

Los métodos de Runge-Kutta pueden, de cierto modo, verse como una generalización del método de Euler, de manera que este podría interpretarse como un Runge-Kutta de orden

1.

3.3.1. Métodos de orden superior

El proceso matemático para obtener la formulación de un RK de orden superior es tedioso, fome y se escapa del alcance del curso, pero de todas formas revisaremos algunos métodos de orden superior, ya que estos son bastante comunes a la hora de resolver EDOs numericamente.

$$K_1 = hf(t, x(t)) \quad (51)$$

$$K_2 = f\left(t + \frac{h}{2}, x(t) + \frac{h}{2}K_1\right) \quad (52)$$

$$K_3 = f\left(t + \frac{h}{2}, x(t) + \frac{h}{2}K_2\right) \quad (53)$$

$$K_4 = f(t + h, x(t) + hK_3) \quad (54)$$

$$x(t + h) = x(t) + \frac{h}{6}(K_1 + 2K_2 + 2K_3 + K_4) \quad (55)$$

Las expresiones (51)-(55) representan el Método de Runge-Kutta de orden 4, o RK4, este esquema es por lejos el más utilizado en la física. Tiene error de orden $\mathcal{O}(h^4)$, y es el último RK cuya cantidad de "pasos" es igual al orden de su error (el RK5 requiere 6 pasos), por esta razón, este método es un balance entre precisión numérica y costo de computación.

$$K_1 = hf(t, x) \quad (56)$$

$$K_2 = hf(t + h, x(t) + K_1) \quad (57)$$

$$K_3 = hf\left(t + \frac{1}{2}h, x(t) + \frac{1}{4}(K_1 + K_2)\right) \quad (58)$$

$$x(t + h) = x(t) + \frac{1}{6}K_1 + \frac{1}{6}K_2 + \frac{2}{3}K_3 \quad (59)$$

Las expresiones (56)-(59) forman una formulación particular de un RK3 denominado **Método de Shu-Osher**. Este esquema mantiene la estabilidad de la solución mejor que el esquema estandar de RK3, por esta razón, se vuelve particularmente útil para resolver problemas con roce, o algún elemento disipativo.

Como hemos visto, los métodos de Runge-Kutta son sumamente útiles debido a su bajo error y relativa simpleza a la hora de implementar computacionalmente, lamentablemente, a diferencia de los esquemas salto de rana, estos no conservan la energía. Esto quiere decir que si usted resuelve un sistema **sin** disipación (e.g. Movimiento planetario, resorte sin amortiguamiento) utilizando métodos RK, la solución que obtendrá va a presentar disipación o introducción de energía no deseada a pesar de que esto difiera de la solución analítica y de la realidad física (en los ejemplos anteriores, el planeta caerá al centro de masa, y/o el resorte incrementará su amplitud hacia el infinito, como muestra la figura 5). Por otro lado, una pequeña ventaja de los RK, es que si la solución de la ecuación a resolver es un

polinomio de grado N , entonces el esquema RKN obtendrá la solución exacta.

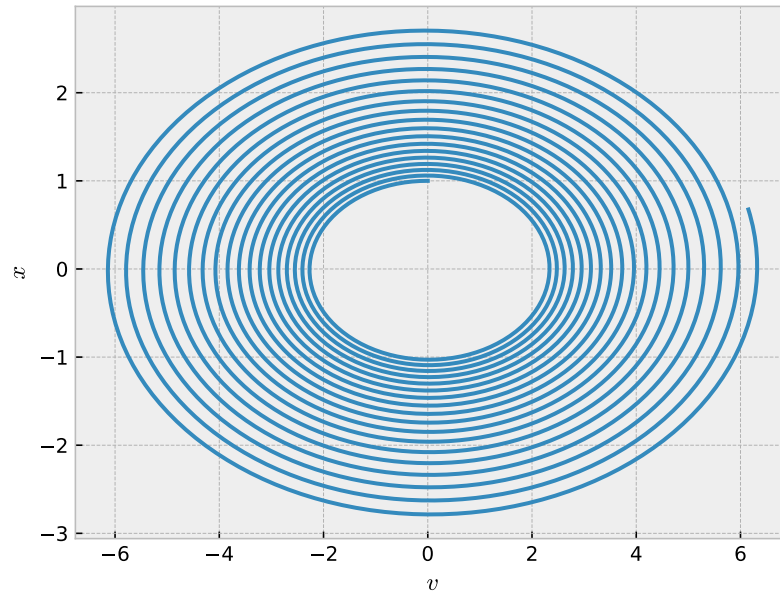


Figura 5: Espacio de fases del oscilador armónico resuelto con RK4.

3.3.2. Formulación semi-general

En general, un método RK de s etapas se puede escribir como

$$K_i = hf \left(t_n + c_i h, y_n + \sum_{j=1}^{i-1} a_{ij} K_j \right), \quad y_{n+1} = y_n + \sum_{i=1}^s b_i K_i \quad (60)$$

donde los coeficientes a_{ij} , b_i y c_i se obtienen mediante un arreglo de valores de la forma (61), denominado **tabla de Butcher**

$$\begin{array}{c|cccc} c_1 & 0 & & & \\ c_2 & a_{21} & 0 & & \\ \vdots & \vdots & \ddots & \ddots & \\ c_s & a_{s1} & \cdots & a_{s,s-1} & 0 \\ \hline & b_1 & b_2 & \cdots & b_s \end{array} \quad (61)$$

Donde la matriz $[a_{ij}]$ se denomina **matriz de Runge-Kutta**, y los b_i y c_i se denominan **pesos** y **nodos**, respectivamente. Por ejemplo, la tabla de Butcher para RK4 es dada por (62)

$$\begin{array}{c|ccc}
0 & & & \\
1/2 & 1/2 & & \\
1/2 & 0 & 1/2 & \\
1 & 0 & 0 & 1 \\
\hline
& 1/6 & 1/3 & 1/3 & 1/6
\end{array} \tag{62}$$

El metodo de Shu-Osher es representado por la tabla (63)

$$\begin{array}{c|cc}
0 & & \\
1 & 1 & \\
1/2 & 1/4 & 1/4 \\
\hline
& 1/6 & 1/3 & 2/3
\end{array} \tag{63}$$

Finalmente, la tabla (64) representa el método de Euler

$$\begin{array}{c|c}
0 & \\
\hline
1 & 1
\end{array} \tag{64}$$

3.3.3. Implementación en Python

3.3.4. Teorema II de Buckingham

Para finalizar esta sección, sería bueno mencionar un teorema importante del análisis dimensional, el **Teorema II de Buckingham**, el cual establece que dado un sistema físico que involucre n magnitudes físicas, representadas por k magnitudes fundamentales, entonces, podemos escribir el sistema por medio de $n - k$ variables adimensionales a partir de las variables originales. A grandes rasgos, este nos dice que los problemas físicos no dependen del sistema de unidades en el que se representen, y además, en algunos casos, nos permite eliminar constantes o parametros del problema, y así determinar si estos son relevantes a la dinámica del problema, o no.

Por ejemplo, consideremos el movimiento vertical de un proyectil considerando la resistencia del aire

$$v'_y = -g - \mu v_y^2, \quad y(0) = y_0, \quad v_y(0) = v_{y0} \tag{65}$$

Para comenzar a adimensionalizar, analizamos las unidades de las variables del problema en la tabla 2

Tabla 2: Variables y sus dimensiones

Variable	t	v_y	v_{y0}	g	μ	y	y_0
Dimensión	T	LT^{-1}	LT^{-1}	LT^{-2}	L^{-1}	L	L

Vemos entonces que tenemos 7 variables en total, expresadas en terminos de 2 unidades fundamentales, longitud (L) y tiempo (T), por lo que, adimensionalizando el sistema, podre-

mos reducir el problema a $7 - 2 = 5$ variables. Ahora, como debemos eliminar **dos** variables, elegimos **dos** constantes con las cuales podamos normalizar (adimensionalizar) el sistema, para ello, notamos que podemos obtener todas las unidades involucradas en el problema multiplicando y dividiendo las constantes v_{y0} y g . Así, definimos nuestro nuevo conjunto de variables adimensionales, denominado Π -grupo⁴, dado por (66)

$$\begin{aligned}\Pi_1 = t \cdot \left(\frac{g}{v_{y0}}\right) = \tau, \quad \Pi_2 = v_y \cdot \left(\frac{1}{v_{y0}}\right) = \nu_y, \quad \Pi_3 = v_{y0} \cdot \left(\frac{1}{v_{y0}}\right) = 1, \quad \Pi_4 = g \cdot \left(\frac{1}{g}\right) = 1, \\ \Pi_5 = \mu \cdot \left(\frac{v_{y0}^2}{g}\right) = \tilde{\mu}, \quad \Pi_6 = y \cdot \left(\frac{g}{v_{y0}^2}\right) = \rho, \quad \Pi_7 = y_0 \cdot \left(\frac{g}{v_{y0}^2}\right) = \rho_0\end{aligned}\tag{66}$$

De esta manera, podemos reescribir el problema original en terminos de nuestras nuevas variables, de modo que

$$\frac{d\nu_y}{d\tau} = -1 - \tilde{\mu}\nu_y^2, \quad \rho(0) = \rho_0, \nu_y(0) = 1\tag{67}$$

De esta manera, podemos deducir que variables son dinámicamente relevantes, con lo cual podemos analizar el problema de manera más eficiente.

Este proceso es útil, pero realmente se utiliza en dinámica de fluidos. De todas maneras, si quieren analizar un problema físico muy complejo numéricamente, es conveniente que normalizar las variables antes de programarlo, ya que, al reducir el número de variables, podemos, potencialmente, ignorar constantes muy pequeñas comparadas al resto de las variables que aumenten el error numérico (e.g. constante gravitacional para movimiento planetario).

3.4. Comentarios Sobre Estabilidad

Motivación

Consideremos un péndulo simple, colgando desde un pivote conectado por un cable ideal (indeformable). Es relativamente fácil ver que la ecuación que rige la cinemática del sistema es:

$$\theta'' + \frac{g}{l} \sin(\theta) = 0,\tag{68}$$

donde $\theta \in (-\pi, \pi]$ es el ángulo que forma el péndulo con la vertical (medido en sentido antihorario), g es la aceleración por fuerza de gravedad y l es la longitud del cable. Generalmente, se suele realizar una aproximación para ángulos pequeños tal que $\sin(\theta) \approx \theta$, de modo que la ecuación a resolver es:

$$\theta'' + \frac{g}{l} \theta = 0,$$

la cual es una ecuación diferencial ordinaria tipo oscilador armónico con soluciones conocidas. Sin embargo, puesto que ya tiene conocimientos para abordar numéricamente estos

⁴ El cual no es un grupo en el sentido matemático :c

problemas, podemos adentrarnos al análisis de problemas como (68) , los cuales no tienen soluciones analíticas. Puesto que ya contamos con las herramientas para aproximar numéricamente su trayectoria, vamos a centrarnos en el análisis de la estabilidad de soluciones, puntos de equilibrio y métodos para abordar estas preguntas.

3.4.1. Criterio de la Derivada

Este criterio rescata directamente las nociones básicas de cálculo en una variable para analizar la **naturaleza de los puntos de una solución**. Consideremos el caso del péndulo simple. Ya sea por medio de un método de búsqueda de ceros o por medio de un análisis físico, es fácil ver que existen dos puntos críticos para el sistema: cuando el péndulo está completamente en reposo ($\theta = 0$), o el caso contrario cuando está verticalmente hacia arriba ($\theta = \pi$). Vale la pena recordar que, para un x_1 tal que $V'(x_1) = 0$ (punto crítico), se tiene :

- Si $V''(x_1) < 0$, entonces x_1 es un punto crítico **asintóticamente estable** (es decir, para tiempos grandes, el sistema tiende a este punto).
- Si $V''(x_1) > 0$, entonces el punto crítico es **inestable** (es decir, dadas condiciones iniciales cercanas a este punto, el sistema tiende a alejarse de él, y dependiendo de la naturaleza del problema, en dirección al punto de equilibrio estable).
- Si $V''(x_1) = 0$, entonces el criterio es no concluyente y requerimos de otro criterio para determinar la naturaleza del punto.

Donde V generalmente es la energía potencial del sistema. En nuestro caso, dada la geometría del problema, es fácil ver que $V(\theta) = -\omega^2 \cos(\theta)$, con $\omega^2 = g/l$. Es fácil chequear que 0 corresponde a un punto de equilibrio estable y π a uno inestable. Generalmente tenemos información respecto del potencial, por lo que la dificultad recae en encontrar donde sus derivadas se anulan. Para esto podemos utilizar el método de la bisección una vez obtenida la primera derivada para obtener los puntos críticos. Luego, podemos ingresarlos a la segunda derivada del potencial para así obtener su clasificación.

3.4.2. Sistemas Autónomos*

3.4.3. Funciones de Lyapunov*

4. Introducción a la Búsqueda de Ceros

Usualmente, cuando trabajamos con ecuaciones, una de nuestras metas es encontrar el valor de cierta variable o también llamada incógnita, para esto, desarrollamos una serie de pasos o "algoritmo" que seguir para encontrar la respuesta. Un buen ejemplo de esto es considerar el caso de una ecuación cuadrática:

$$ax^2 + bx + c = 0 \quad (69)$$

Es bien sabido que, dado los coeficientes a , b y c , existe una fórmula general que seguir, la conocida como ecuación cuadrática o *Ecuación de Bashkara*:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \quad (70)$$

Al introducir los respectivos valores de los coeficientes, podemos encontrar las dos soluciones a este problema.

Sin embargo, no siempre se puede encontrar una ecuación para los valores de incógnitas en toda ecuación, un ejemplo a esto es el siguiente:

$$\sin x + x = 0 \quad (71)$$

A pesar de lo simple de nuestro problema (puesto que es la suma entre una función trigonométrica y la incógnita), ya no es posible encontrar una ecuación de forma analítica que exprese valores exactos de la soluciones para "x", así, la pregunta es ¿Qué podemos hacer en estos casos?

Para esto, se desarrollan los **Métodos para hallar ceros de una función**

Estos, se basan en una idea principal, convertir la ecuación en la forma:

$$f(x) = 0 \quad (72)$$

Esto, con la finalidad de que ahora nuestro trabajo, será encontrar el valor para el cual, "x" cumple con ser un "cero" de la función.

Para encontrar este cero, es importante establecer un "intervalo" en donde estemos trabajando (puesto que la computadora no es capaz de analizar, en caso de trabajar en $\mathbb{R}$, la recta $] -\infty, +\infty[$, ya que es infinita), para esto, tenemos que considerar un intervalo en donde deba existir una solución.

Sin embargo, ¿Cómo podemos asegurar una solución?

Para ello, existe el **Teorema de Bolzano**, el cual establece que:

Para una función continua en un intervalo cerrado $[a, b]$, tal que $f(a) \cdot f(b) < 0$, entonces debe existir $c \in [a, b]$ tal que $f(c) = 0$

Así, ya podemos concluir que el método con el que trabajemos esté siendo usado en un intervalo donde la solución exista.

Es importante considerar que debido a que utilizaremos métodos numéricos, la solución

que encontraremos nunca será del todo correcta, sino, tan solo una aproximación a la respuesta real o analítica.

Por ello, es importante tener en cuenta *¿Qué tan cerca queremos o podemos estar de nuestra respuesta real?*. Para esto, es importante tener en cuenta el uso de una medida de "Tolerancia".

La tolerancia, se puede definir como el intervalo más pequeño sobre el cual vamos a realizar el algoritmo de búsqueda de nuestro cero. Cuando nuestros intervalos sean suficientemente pequeños, le indicaremos al programa que detenga el algoritmo y se quede con el valor o respuesta dentro del intervalo encontrado.

Así, si la tolerancia (denotada como δ) cumple que: $f(x_i) < \delta$, entonces la solución numérica encontrada será x_i .

Esto conlleva a la cuestión *¿Cómo podemos medir el error que se encuentra dentro de nuestro método?* Para esto, es importante analizar *qué tan rápido aumenta el error*, tema del cual se verá dentro de cada método.

A continuación, exploraremos algunos de los métodos para encontrar ceros.

4.1. Método de la Bisección

Comúnmente conocido bisección solamente, es el método más fácil de generar (y a su vez, el de explicar)

Consideremos una ecuación de la forma:

$$f(x) = 0 \tag{73}$$

Donde sabemos por *Teorema de Bolzano* que existe una solución en el intervalo $[a, b]$.

Para realizar el método de la bisección, dividiremos el intervalo en dos partes, sea $c = \frac{a+b}{2}$, así, ahora existen dos intervalos, $[a, c]$ y $[c, b]$.

Ahora, analizaremos cada intervalo por separado a través del *Teorema de Bolzano*, esto es:

- Si $f(a) \cdot f(c) < 0$: La solución debe encontrarse en este intervalo, por lo que nos centraremos en trabajar aquí.
- Si $f(a) \cdot f(c) = 0$: La solución está en uno de los límites de los intervalos, como en el proceso anterior el resultado fue negativo, entonces podemos confirmar que la solución es $x = c$
- Si $f(a) \cdot f(c) > 0$: La solución no está dentro de este intervalo, así, debemos trabajar con el intervalo siguiente $([c, b])$.

Una vez realizado este análisis, encontraremos la solución o bien, el intervalo donde se encuentra. Siguiendo esto, podemos seguir realizando el mismo algoritmo para encontrar intervalos más pequeños en donde nuestra solución se debe encontrar, esto, hasta que la función evaluada en el valor medio (El valor c que generamos al dividir el intervalo) sea menor a la Tolerancia con la que estamos trabajando.

De esta forma, podemos catalogar el método de la bisección de forma general:

Considerar una ecuación de la forma $f(x) = 0$ en el intervalo $[a, b]$

- Analizamos el producto de la función $f(x)$ evaluada en los límites ($x = a, b$), esto es $f(a) \cdot f(b)$
- Si el producto es negativo, el valor "x" se encuentra dentro de este intervalo, por lo que podemos continuar con el procedimiento.
- Dividimos el intervalo en dos partes, sea $[a, c]$, $[c, b]$
- Analizamos el producto de la función evaluada en los límites del primer intervalo
- Si este producto es negativo, la solución debe encontrarse en este intervalo, por lo que dividimos y volvemos a aplicar el mismo análisis
- Repetimos este proceso hasta que la función evaluada en el punto medio sea menor a nuestra tolerancia.

Podemos observar un ejemplo de este a través del siguiente código:

```

1 f = lambda x: np.cos(x)
2 tolerancia = 1e-10
3 i=0
4 while abs(a-b) > tolerancia:
5     i += 1
6     if f(a)*f(b) < 0:
7         c = (b+a)/2
8         if f(a)*f(c) < 0:
9             b = c
10        else:
11            a = c
12 print("El cero encontrado es", c, "con", i, "iteraciones.")

```

Para nuestro caso, hemos de elegir la función $\cos(x)$, si consideramos el intervalo $[0, 2]$, podemos encontrar el cero que se posiciona en $x = \frac{\pi}{2}$, donde bajo este código, obtenemos un valor de $x \approx 1.5707963$, bastante cercano al buscado.

Finalmente, es importante considerar que este método no considera particularidades de la función $f(x)$, en consecuencia, el tiempo o iteraciones que conlleve será dependiente únicamente del cálculo numérico y la tolerancia que se le agregue.

Ahora bien, **¿Cuál es la proporción de error que existe en este método?**

4.2. Método de Newton

¿Qué ocurre si deseamos utilizar información de la propia función?

Consideraremos buscar el cero de nuestra función $f(x)$ a través del uso de la serie de Taylor.

Para esto, tomemos en nuestro algoritmo la función $f(x)$, si desarrollamos en segundo orden la serie de Taylor para el cero, se puede notar que:

$$f(x_0) = f(x) + (x - x_0) \cdot f'(x) + (x - x_0)^2 \cdot \frac{f''(x)}{2} + \dots \quad (74)$$

Pero por definición, la función evaluada en nuestro punto x_0 debe ser cero, así, si aproximamos la serie:

$$0 \approx f(x) + (x - x_0) \cdot f'(x) \quad (75)$$

Podemos despejar así x_0 , donde:

$$x_0 = x - \frac{f(x)}{f'(x)} \quad (76)$$

Esto, nos ofrece un algoritmo que seguir, a modo de poder aproximarse a través de iteraciones hasta superar la tolerancia establecida.

Notar que a diferencia del método de la bisección, aquí se hace uso de la derivada, la cual es útil para determinar con mayor rapidez el cero con respecto a la tolerancia establecida, dando así un método eficiente y que hace uso de menores iteraciones con respecto a la bisección.

Consideraremos un punto inicial, tal como el valor inicial del intervalo dentro del cual estamos buscando el cero, para este caso, será x_n , así, la iteración del algoritmo nos dará un valor cercano al cero, siendo este denotado x_{n+1} , así:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \quad (77)$$

Un ejemplo de la implementación de este algoritmo es el siguiente, consideramos una función $f(x) = x^3 - 5x$, y su derivada $f'(x) = 3x^2 - 5$, así:

```

1 tolerancia = 1e-12
2
3 f = lambda x: x**3 - 5*x # Se define la función a utilizar
4 fprima = lambda x: 3*x**2 - 5
5
6 def met_newton(f,a,b,tolerancia):
7     zeros = []
8     int = np.linspace(a,b,10)
9     print(int)
10    for i in range(int.size-1):
11
12        if f(int[i])*f(int[i+1]) <= 0:
13            a,b = int[i:i+2]
14            fa, fprima_a = f(a), fprima(a)
15            m = 0
16            while (abs(fa/fprima_a)) >= tolerancia:
17                b = a - fa/fprima_a

```

```

18         a = b
19         fa, fprima_a = f(a), fprima(a)
20         m += 1
21         print("Existe un cero",b,"con iteraciones de",m,"veces")
22
23
24 met_newton(f,-3,3,tolerancia)

```

Notar que si la derivada se acerca a cero, la cantidad de iteraciones comenzará a aumentar, puesto que la velocidad con la que avanza el algoritmo comienza a disminuir.

A su vez, es importante mencionar el error que conlleva este método, notar que de (75), aproximamos en segundo orden, así, el término $O(\delta x^2)$ desaparece de la serie de Taylor. Es importante mencionar esto puesto que, si bien el método converge de forma rápida, muchas veces podría no hacerlo debido a que los términos de ordenes superiores comienzan a generar diferencias notables con respecto a la aproximación hecha

4.3. Método de la Secante

Nota del autor: Este capítulo lo hice en el portafolio de física compu 2

¿Qué ocurre si no podemos calcular u obtener la derivada de la función?

Esto es común en problemas donde la derivada puede no existir en ciertas partes, por lo que es importante analizar casos de este estilo.

Para esto, nace el método de la secante, el cual se basa en la idea de aproximar la derivada de la función, de tal forma que, si consideramos diferencias lo suficientemente pequeñas entre x_n y x_{n-1} , entonces:

$$f'(x_n) \approx \frac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}} \quad (78)$$

Por lo que el algoritmo a iterar constaría de la siguiente estructura:

$$x_{n+1} = x_n - \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})} f(x_n) \quad (79)$$

Un ejemplo al utilizar este método es el siguiente:

```

1
2 tolerancia = 1e-5
3
4 f = lambda x: x**3 - 5*x # Se define la función a utilizar
5
6 # Se define el método de la secante
7 def met_secante(f,a,b,tolerancia):
8     zeros = []
9     int = np.linspace(a,b,10)
10    print(int)
11    for i in range(int.size-1):

```

```
12
13     if f(int[i])*f(int[i+1]) <= 0:
14         a,b = int[i:i+2]
15         fa,fb = f(a), f(b)
16         m = 0
17         while (abs(fb*(b-a)/(fb-fa)) >=tolerancia):
18             bold=b
19             b = b - fb*(b-a)/(fb-fa)
20             fa=fb
21             a=bold
22             fb=f(b)
23             m +=1
24             print("Existe un cero",b,"con iteraciones de",m,"veces")
25             zeros.append(b)
26     return zeros
27
28 met_secante(f,-3,3,tolerancia)
```

Ahora bien, es importante considerar **¿Cuál es la proporción de error dentro del método de la secante?**

5. Interpolación y ajustes

Como mencionamos muy anteriormente, muchas veces no tendremos el lujo de lidiar con funciones definidas analíticamente en todos sus puntos, en lugar de eso, tendremos algún conjunto discreto de puntos $(x_i, f_i)_{i=0, \dots, N}$. En estos casos podríamos querer hacer una de dos cosas: interpolar, o bien, ajustar.

5.1. Interpolación de Lagrange

La interpolación consiste en encontrar una función que pase por todos y cada uno de los puntos dados. En estricto rigor estamos llenando el espacio entre medio de los puntos. Si le suena familiar, es porque esto es lo que hacen los modelos de lenguaje (Gemini, ChatGPT, Claude, etc); la diferencia es que estos lo hacen con sets de datos enormes. Ahora, dados un conjunto arbitrario de puntos $(x_i, f_i)_{i=0, \dots, N}$ siempre podemos encontrar un polinomio de grado menor o igual a N que interpole los puntos, es decir

$$f_i = p(x_i) \quad (80)$$

Podemos proponer una combinación lineal de polinomios de la forma

$$L(x) = \sum_{j=0}^N f_j \ell_j(x) \quad (81)$$

imponiendo la condición

$$\ell_j(x_i) = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases} \quad (82)$$

con tal que se cumpla (80). Podemos encontrar que los polinomios $\ell_j(x)$, denominados **polinmios base de Lagrange**, son dados por:

$$\ell_j(x) = \left(\frac{x - x_0}{x_j - x_0} \right) \left(\frac{x - x_1}{x_j - x_1} \right) \dots \left(\frac{x - x_{j-1}}{x_j - x_{j-1}} \right) \left(\frac{x - x_{j+1}}{x_j - x_{j+1}} \right) \dots \left(\frac{x - x_N}{x_j - x_N} \right) \quad (83)$$

$$= \prod_{i=0, i \neq j}^N \left(\frac{x - x_i}{x_j - x_i} \right) \quad (84)$$

y forman (81), denominado **polinomio interpolador de Lagrange**.

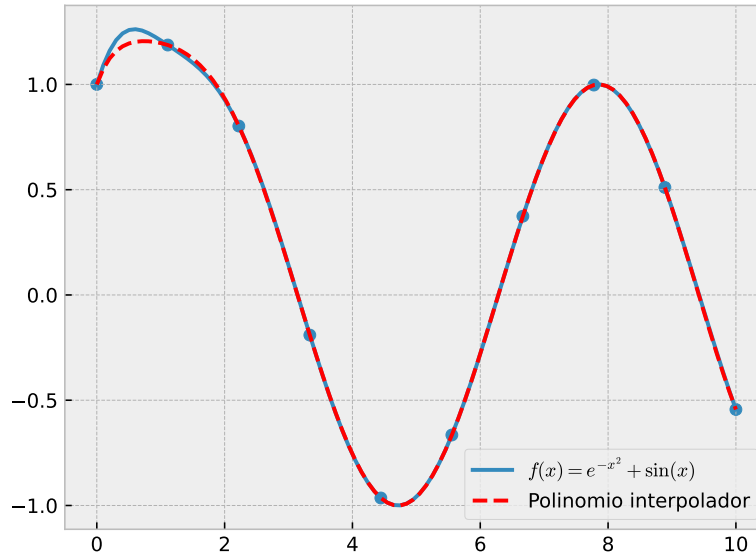


Figura 6: Conjunto de puntos interpolados por Lagrange y función original.

5.2. Matriz de Vandermonde

En la sección anterior, logramos interpolar un conjunto de puntos utilizando una combinación lineal de polinomios (vea (81)), por lo que es natural preguntarse si existe algún otro tipo de función interpoladora. La respuesta es que podemos interpolar con cualquier tipo de función que forme una **base del espacio de funciones**. Consideremos que queremos interpolar un conjunto de puntos (x_j, f_j) por medio de una función $p(x)$ tal que

$$f_j = p(x_j)$$

tenemos que

$$p(x) = \sum_{i=0}^N p_i \varphi_i(x) \quad (85)$$

$$= p_0 \varphi_0(x) + p_1 \varphi_1(x) + \cdots + p_N \varphi_N(x) \quad (86)$$

Así, tenemos el sistema de ecuaciones

$$f_j = p_0 \varphi_0(x_j) + p_1 \varphi_1(x_j) + \cdots + p_N \varphi_N(x_j) \quad (87)$$

donde queremos encontrar los p_j . Matricialmente tenemos que

$$\begin{bmatrix} \varphi_0(x_0) & \varphi_1(x_0) & \cdots & \varphi_N(x_0) \\ \varphi_0(x_1) & \varphi_1(x_1) & \cdots & \varphi_N(x_1) \\ \vdots & \vdots & \ddots & \vdots \\ \varphi_0(x_N) & \varphi_1(x_N) & \cdots & \varphi_N(x_N) \end{bmatrix} \begin{bmatrix} p_0 \\ p_1 \\ \vdots \\ p_N \end{bmatrix} = \begin{bmatrix} f_0 \\ f_1 \\ \vdots \\ f_N \end{bmatrix} \quad (88)$$

De esta manera, reducimos el problema a resolver el sistema encontrando la inversa de la matriz $\varphi_i(x_j)$. Finalmente, todo este proceso es una generalización de encontrar los coeficientes de una serie que aproxime una función, y es analogo a encontrar los coeficientes de una serie de Taylor. El tipo de serie por el cual estemos interpolando depende de como elijamos la función $\varphi_n(x)$, por ejemplo

- Serie de Fourier: $\varphi_n(x) = \exp^{in\pi x/L}$
- Serie de polinomios de Lagrange: $\varphi_n(x) = \ell_n(x)$
- Serie de Legendre: $\varphi_n(x) = P_n(x)$ (Polinomios de Legendre)

5.2.1. Casos particulares interesantes

Si elegimos los polinomios de grado n como nuestra base de funciones, es decir, $\varphi_n(x) = x^n$, la matriz del lado izquierdo de (88) se convierte en la matriz (89)

$$\begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^N \\ 1 & x_1 & x_1^2 & \cdots & x_1^N \\ \vdots & \vdots & \ddots & \vdots & \\ 1 & x_N & x_N^2 & \cdots & x_N^N \end{bmatrix} \quad (89)$$

denominada **Matriz de Vandermonde**.

Alternativamente, si hacemos que $\varphi_n(x) = \ell_n(x)$, por la propiedad (82), la matriz izquierda de (88) se convierte en la matriz identidad de $N \times N$, lo que reafirma la condición (80).

5.3. Splines

Si interpolamos un conjunto de puntos con una sola función para todo el rango de los puntos, podemos encontrarnos con el **fenómeno de Gibbs**, como muestra la figura 7. Esto significa que la función interpoladora oscila o alcanza valores muy altos entre dos puntos adyacentes.

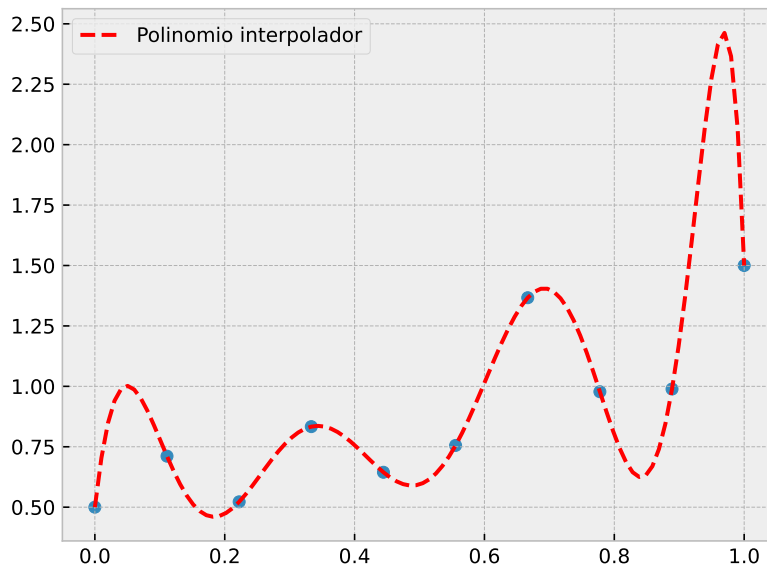


Figura 7: Fenómeno de Gibbs causado por interpolación de Lagrange.

Existen varias maneras de mitigar este fenómeno, como utilizar nodos no equidistantes (ver **Nodos de Gauss-Chebyshev**), o bien, si no queremos modificar los puntos, podemos realizar una interpolación "por partes", es decir, podemos interpolar el espacio entre subconjuntos de puntos exigiendo que todas las interpolaciones sean continuas y suficientemente⁵ derivables entre conjuntos de puntos adyacentes, es decir, que la unión de las interpolaciones sea suave. En general, dado un conjunto de puntos (x_i, y_i) , una **spline** será un polinomio de grado k , definido por tramos, que debe ser $k - 1$ veces continuamente diferenciable. En la práctica, se suelen utilizar *cubic splines*, es decir, debemos interpolar cada subintervalo de nuestro intervalo con un polinomio de grado 3, el cual debe ser continuo y dos veces continuamente diferenciable en los nodos.

Por ejemplo, consideremos que queremos interpolar un conjunto de datos $[x_1, x_2, x_3]$, $[y_1, y_2, y_3]$ utilizando cubic splines. Significa que queremos interpolar los subintervalos $[x_1, x_2]$ y $[x_2, x_3]$ con polinomios de grado 3, esto es

$$p_1(x) = a_1x^3 + a_2x^2 + a_3x + a_4, \quad p_2(x) = b_1x^3 + b_2x^2 + b_3x + b_4 \quad (90)$$

Ahora, si imponemos nuestras condiciones de que las interpolaciones sean continuas y dos

⁵ El significado de "suficiente" dependerá del tipo de spline que queramos utilizar.

veces diferenciables, tendremos:

$$p_1(x_1) = a_1x_1^3 + a_2x_1^2 + a_3x_1 + a_4 = y_1, \quad p_2(x_2) = b_1x_2^3 + b_2x_2^2 + b_3x_2 + b_4 = y_2 \quad (91)$$

$$p_1'(x_2) = p_2'(x_2), \quad 3a_1x_2^2 + 2a_2x_2 + a_3 = 3b_1x_2^2 + 2b_2x_2 + b_3 \quad (92)$$

$$p_1''(x_2) = p_2''(x_2), \quad 6a_1x_2 + 2a_2 = 6b_1x_2 + 2b_2 \quad (93)$$

Ahora, vemos que tenemos 8 variables, y solo 6 ecuaciones, por lo que nuestras condiciones aún son insuficientes. Para completar el sistema debemos introducir una última condición sobre el valor de la segunda derivada en los extremos (x_1 y x_3), en estricto rigor podemos elegir cualquier valor, pero generalmente se escoge que

$$p_1''(x_1) = 0, \quad p_2''(x_3) = 0 \quad (94)$$

Las ecuaciones (94) se denominan **condición natural**, y al imponerla nuestra spline se denomina una **spline natural**.

5.4. Ajustes

A diferencia de una interpolación, un ajuste busca la función que más se acerque a todos los puntos de un conjunto de puntos, es decir, la curva que mejor represente la tendencia de la distribución de puntos. Si por alguna razón sabemos que los datos deberían seguir una tendencia en específico, podemos establecer un modelo y ajustar los parámetros del mismo para encontrar los valores de estos que se ajusten mejor a los datos.

5.4.1. Regresión Lineal

Comenzamos con el modelo más simple, dado un conjunto de puntos (x_i, y_i) , buscamos los valores para los parámetros α_0 y α_1 tal que la curva de la forma (95) se ajusta mejor a los puntos.

$$y = \alpha_0 + \alpha_1 x \quad (95)$$

Para ello, notamos que, bajo este modelo, podemos escribir una relación para x_i e y_i como

$$y_i = \alpha_0 + \alpha_1 x_i + \epsilon_i \quad (96)$$

donde ϵ_i es el error entre el valor predicho por el modelo, y el dato real, denominado **residuo**. Ahora, la curva que mejor se ajuste a los datos debería también minimizar el valor de los residuos, existen muchas maneras de cuantificar esto, pero nosotros utilizaremos el método de mínimos cuadrados, de modo que debemos minimizar la función χ^2 , definida por

$$\chi^2 = \sum_{i=1}^N \epsilon_i^2 = \sum_{i=1}^N (y_i - \alpha_0 - \alpha_1 x_i)^2 \quad (97)$$

Como estamos variando los **parámetros** α_0 y α_1 , tenemos que $\chi^2 = \chi^2(\alpha_0, \alpha_1)$

$$\frac{\partial^2 \chi^2}{\partial \alpha_0^2} = -2 \sum_{i=1}^N (y_i - \alpha_0 - \alpha_1 x_i) = 0, \quad (98)$$

$$\frac{\partial^2 \chi^2}{\partial \alpha_1^2} = -2 \sum_{i=1}^N (y_i - \alpha_0 - \alpha_1 x_i) x_i = 0 \quad (99)$$

de modo que, distribuyendo la sumatoria, obtenemos el sistema de ecuaciones lineales

$$N\alpha_0 + \left(\sum_{i=1}^N x_i \right) \alpha_1 = \sum_{i=1}^N y_i \quad (100)$$

$$\left(\sum_{i=1}^N x_i \right) \alpha_0 + \left(\sum_{i=1}^N x_i^2 \right) \alpha_1 = \sum_{i=1}^N y_i x_i \quad (101)$$

Para ahorrar un poco de notación, dividimos ambas ecuaciones por N , y notamos que el promedio de un conjunto de N elementos es dado por $\bar{x} = \sum_{i=1}^N x_i / N$. Así,

$$\alpha_0 + \bar{x}\alpha_1 = \bar{y} \quad (102)$$

$$\bar{x}\alpha_0 + \overline{x^2}\alpha_1 = \overline{xy} \quad (103)$$

⁶ matricialmente,

$$\begin{pmatrix} 1 & \bar{x} \\ \bar{x} & \overline{x^2} \end{pmatrix} = \begin{pmatrix} \bar{y} \\ \overline{xy} \end{pmatrix} \quad (104)$$

De este modo, reducimos el problema a resolver un sistema de ecuaciones lineales. En general, la solución al sistema (104) será

$$\alpha_0 = \bar{y} - \alpha_1 \bar{x}, \quad \alpha_1 = \frac{\overline{xy} - (\bar{x})(\bar{y})}{\overline{x^2} - \bar{x}^2} \quad (105)$$

Pero generalmente no es conveniente resolver el sistema de forma analítica, en lugar de eso, podemos utilizar **python**! En efecto,

Código 11: Resolución del sistema lineal (104) en python.

```
1 import numpy as np
2
3 #datos 100 % inventados, pero podrían ser el resultado de un experimento!
4 xi = np.array([1, 2, 3, 4, 5])
5 yi = np.array([2.5, 3.8, 6.7, 7.9, 10.1]) #tendencia casi lineal
6
```

⁶ Notar que $\overline{x^2} \neq \bar{x}^2$, el primero es el **promedio de los cuadrados**, y el último el **cuadrado de los promedios**

```

7 #Definimos las variables necesarias
8 xprom = np.mean(xi) #promedio de los x
9 yprom = np.mean(yi) #promedio de los y
10
11 x2prom = np.mean(xi**2) #promedio de los  $x^2$ 
12 xyprom = np.mean(xi*yi) #promedio de los  $x*y$ 
13
14 #definimos la matriz de parametros (lado izquierdo de (95))
15 A = np.array([[1, xprom],
16              [xprom, x2prom]])
17
18 #definimos la matriz de terminos "libres" (lado derecho de (95))
19 B = np.array([yprom,
20              [xyprom]])
21
22 a0, a1 = np.linalg.solve(A, B) #utilizamos linalg.solve para resolver el sistem con numpy!
23

```

Graficando los puntos del código 11, y la recta definida por los parámetros obtenidos al resolver el sistema, obtenemos la figura 8.

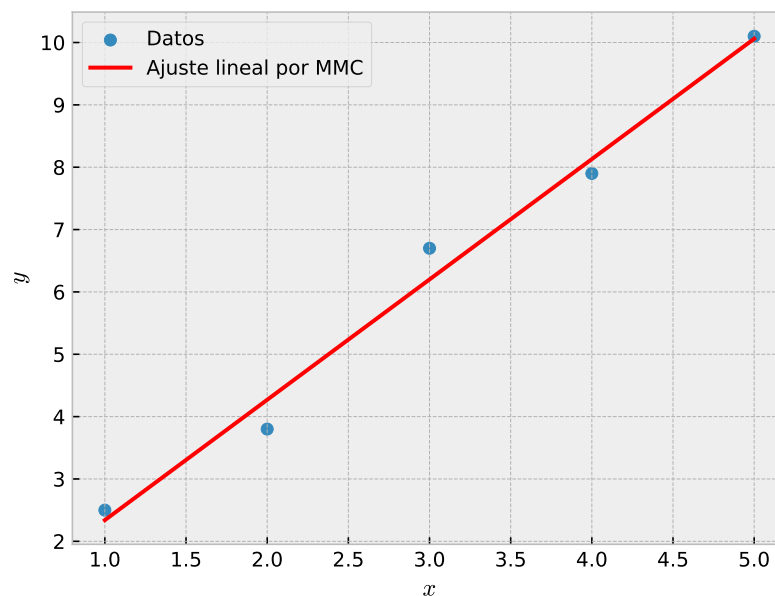


Figura 8: Ajuste lineal para distribución de puntos.

De este modo, hemos encontrado la recta que mejor se ajusta a la tendencia de los datos. Ahora, podemos generalizar este proceso para un ajuste polinomial de grado n , en este caso

la función χ^2 tendrá la forma

$$\chi^2 = \sum_{i=1}^N (y_i - f(x_i))^2 = \sum_{i=1}^N (y_i - a_0 - a_1 x_i - a_2 x_i^2 - \cdots - a_n x_i^n) \quad (106)$$

Si derivamos con respecto a cada a_j e igualamos cada derivada a 0, obtendremos el sistema

$$\begin{pmatrix} N & \sum x_i & \sum x_i^2 & \cdots & \sum x_i^n \\ \sum x_i & \sum x_i^2 & \sum x_i^3 & \cdots & \sum x_i^{n+1} \\ \sum x_i^2 & \sum x_i^3 & \sum x_i^4 & \cdots & \sum x_i^{n+2} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \sum x_i^n & \sum x_i^{n+1} & \sum x_i^{n+2} & \cdots & \sum x_i^{2n} \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_n \end{pmatrix} = \begin{pmatrix} \sum y_i \\ \sum x_i y_i \\ \sum x_i^2 y_i \\ \vdots \\ \sum x_i^n y_i \end{pmatrix} \quad (107)$$

Si resolvemos el sistema (107), obtendremos los valores de los parámetros del polinomio de grado n que se ajusta mejor a la distribución de puntos.

5.4.2. Método de mínimos cuadrados ponderado

6. Integración numérica

6.1. Introducción

Continuando con la motivación de este apunte, necesitamos evaluar numéricamente la integral de una función cualquiera (continua en el intervalo $a < x < b$), para ello se puede aproximar mediante la suma de áreas (rectángulos, trapezoides, etc) e interpolación, como veremos a continuación.

6.2. Método Newton-Cotes

El método Newton-Cotes consiste en evaluar la integral como la suma de áreas (rectángulos, trapezoides, etc) en subintervalos. Considerando una cantidad de n datos discretos (o nodos) y suponiendo $(x_i, f(x_i))$ son conocidos, con $i = 0, 1, 2, \dots, n-1, n$ en un intervalo $[a, b]$ con $x_0 = a$ y $x_n = b$, es decir: $a \leq x_i < \dots < x_n \leq b$. Donde para subintervalos equiespaciados x_i corresponde al i -ésimo punto a evaluar: $x_i = a + ih$ y $h = \frac{b-a}{n-1}$

Estas áreas se pueden obtener mediante Interpolación, teniendo que de manera general:

$$\int_a^b f(x)dx \approx \sum_{i=0}^{n-1} w_i f(x_i) = \sum_{i=0}^{n-1} \left(\int_a^b \ell_i(x)dx \right) f(x_i) \quad (108)$$

donde w_i son los pesos, que corresponden a la integral de los polinomios base de Lagrange (Ver capítulo de Interpolación).

6.3. Regla del rectángulo

La primera idea que se viene a la mente al tratar de aproximar una integral es mediante el área de un rectángulo:

$$\int_a^b f(x)dx \approx \frac{b-a}{2} f(a) = hf(a)$$

claramente no es para nada preciso, por lo que subdividiremos el intervalo $[a, b]$ en $n-1$ subintervalos y sumaremos el área de cada rectángulo (regla compuesta):

$$\int_a^b f(x)dx = \sum_{i=0}^{n-1} f(x_i)(x_{i+1} - x_i)$$

con $h_i = x_{i+1} - x_i$ el ancho, ahora si son equiespaciados entonces:

$$\int_a^b f(x)dx = \sum_{i=0}^{n-1} \frac{b-a}{n-1} f(x_i) = \sum_{i=0}^{n-1} hf(x_i)$$

También se puede ver como una interpolación de grado 0, con esto en mente podemos calcular el error expandiendo en Serie de Taylor alrededor de x_i hasta primer orden, teniendo así:

$$\begin{aligned}
f(x) &\approx f(x_i) + f'(\xi)(x - x_i) \\
\int_{x_i}^{x_{i+1}} f(x)dx &= \int_{x_i}^{x_{i+1}} f(x_i)dx + \int_{x_i}^{x_{i+1}} f'(\xi)(x - x_i)dx \\
&= hf(x_i) + \int_0^{x_{i+1}-x_i} f'(\xi)udu \\
&= hf(x_i) + \frac{1}{2}h^2 f'(\xi)
\end{aligned}$$

donde se realizó el cambio de variable $u = x - x_i$ y el ancho del intervalo $h = x_{i+1} - x_i$, y donde el error corresponde a:

$$\mathcal{O}(h^2) = \frac{1}{2}h^2 f'(\xi)$$

teniendo así que:

$$\int_{x_i}^{x_{i+1}} f(x)dx = hf(x_i) + \mathcal{O}(h^2)$$

tomando en cuenta la integral original, tenemos:

$$\int_a^b f(x)dx = \sum_{i=0}^{n-1} hf(x_i) + \mathcal{O}(nh^2)$$

6.4. Regla del trapecoide

Otra forma más precisa es aproximar la curva mediante el área de un trapecoide, para $f(x)$ con $x_i < x < x_{i+1}$, se tiene:

$$\int_{x_i}^{x_{i+1}} f(x)dx \approx (x_{i+1} - x_i)f(x_i) + \frac{1}{2}(x_{i+1} - x_i)(f(x_{i+1}) - f(x_i)) = \frac{h}{2}(f(x_i) + f(x_{i+1}))$$

regla compuesta

$$\int_a^b f(x)dx = \sum_{i=0}^{n-1} \int_{x_i}^{x_{i+1}} f(x)dx \approx \frac{h}{2}f(x_0) + hf(x_1) + \dots + hf(x_{n-2}) + \frac{h}{2}f(x_{n-1})$$

error

$$f(x) \approx f(x_i) + f'(x - x_i) + \frac{1}{2}f''(x_i)(x - x_i)^2$$

6.5. Regla de Simpson

Para este caso se hace una interpolación de $f(x)$ de grado 2

$$\int_{x_0}^{x_1} f(x)dx \approx \frac{h}{3}[f(x_0) + 4f(x_M) + f(x_1)]$$

con $x_M = \frac{x_1+x_0}{2}$